Chapter 6: Performance Considerations 4

CSCD439/539 GPU Programming

Performance Considerations

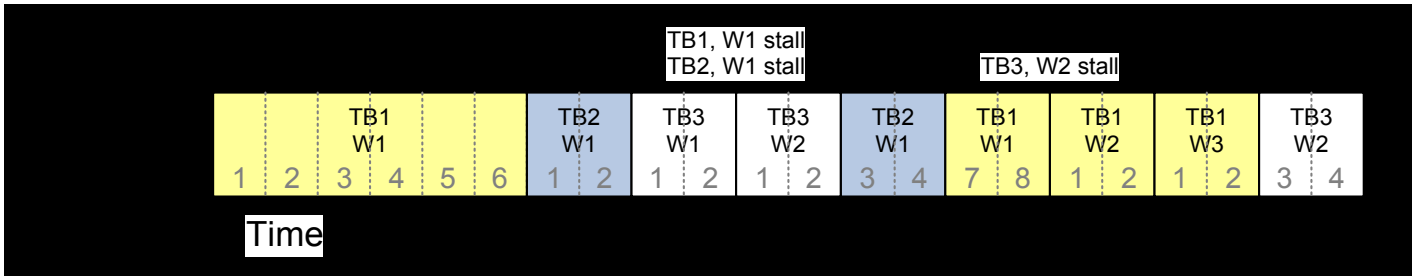
- How SM Execute Threads
- Control-Flow Divergence
- Occupancy
- Kernel Launch Overheads

Number of threads that can reside in a SM

OCCUPANCY

Reminder: Thread Scheduling

- SM implements zero-overhead warp scheduling
  - For old GPUs, at any time, only one of the warps is executed by SM *(not true for new gpus)
  - Warps whose next instruction has its inputs ready for consumption are eligible for execution
  - Eligible Warps are selected for execution on a prioritized scheduling policy
  - All threads in a warp execute the same instruction when selected



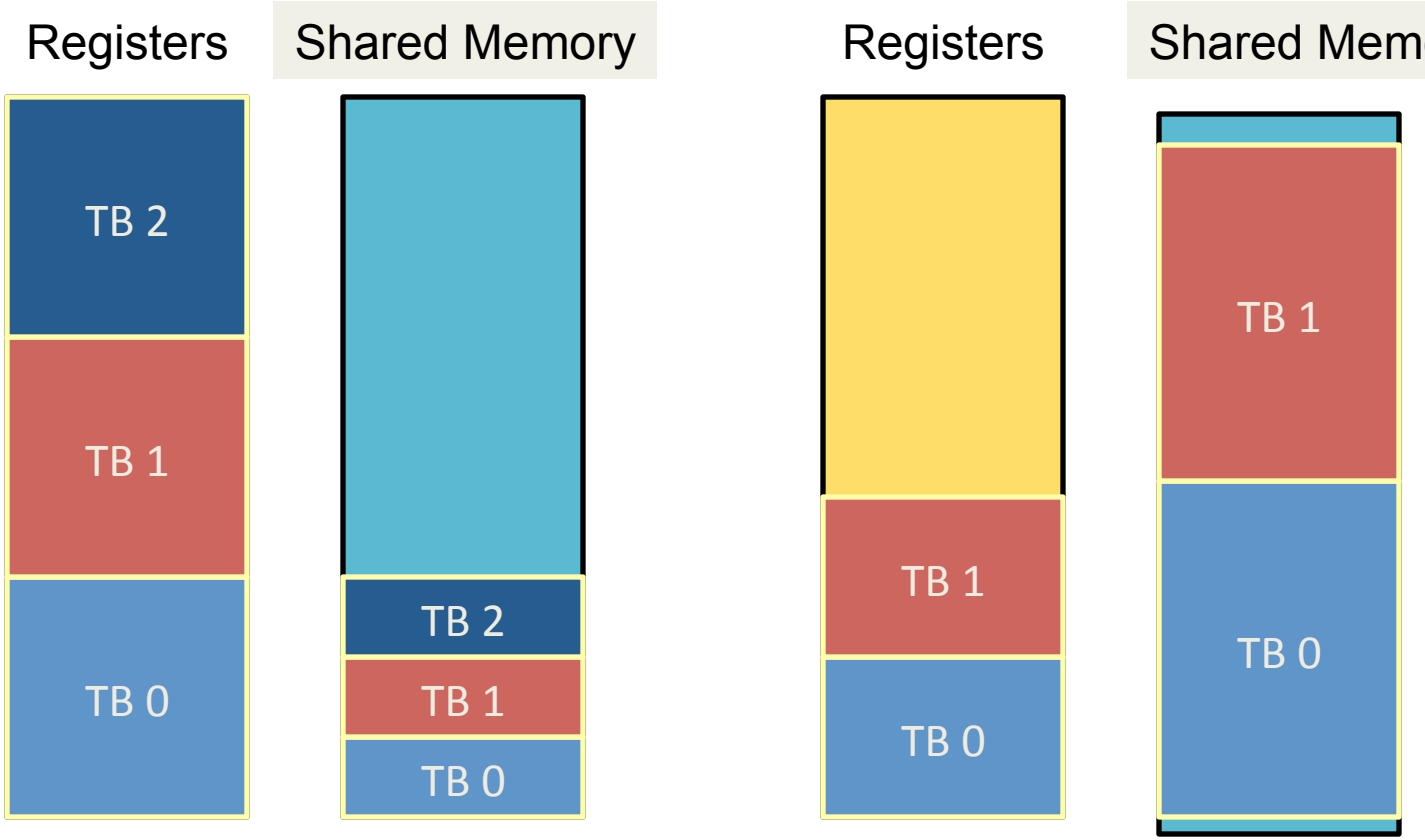
Thread Scheduling

- What happens if all warps are stalled?
  - No instruction issued → performance lost
- Most common reason for stalling?
  - Waiting on global memory
- If your code reads global memory every couple of instructions
  - You should try to maximize occupancy
  - Why?

What determines occupancy?

- Register usage per thread & shared memory per thread block

Resource Limits (1)



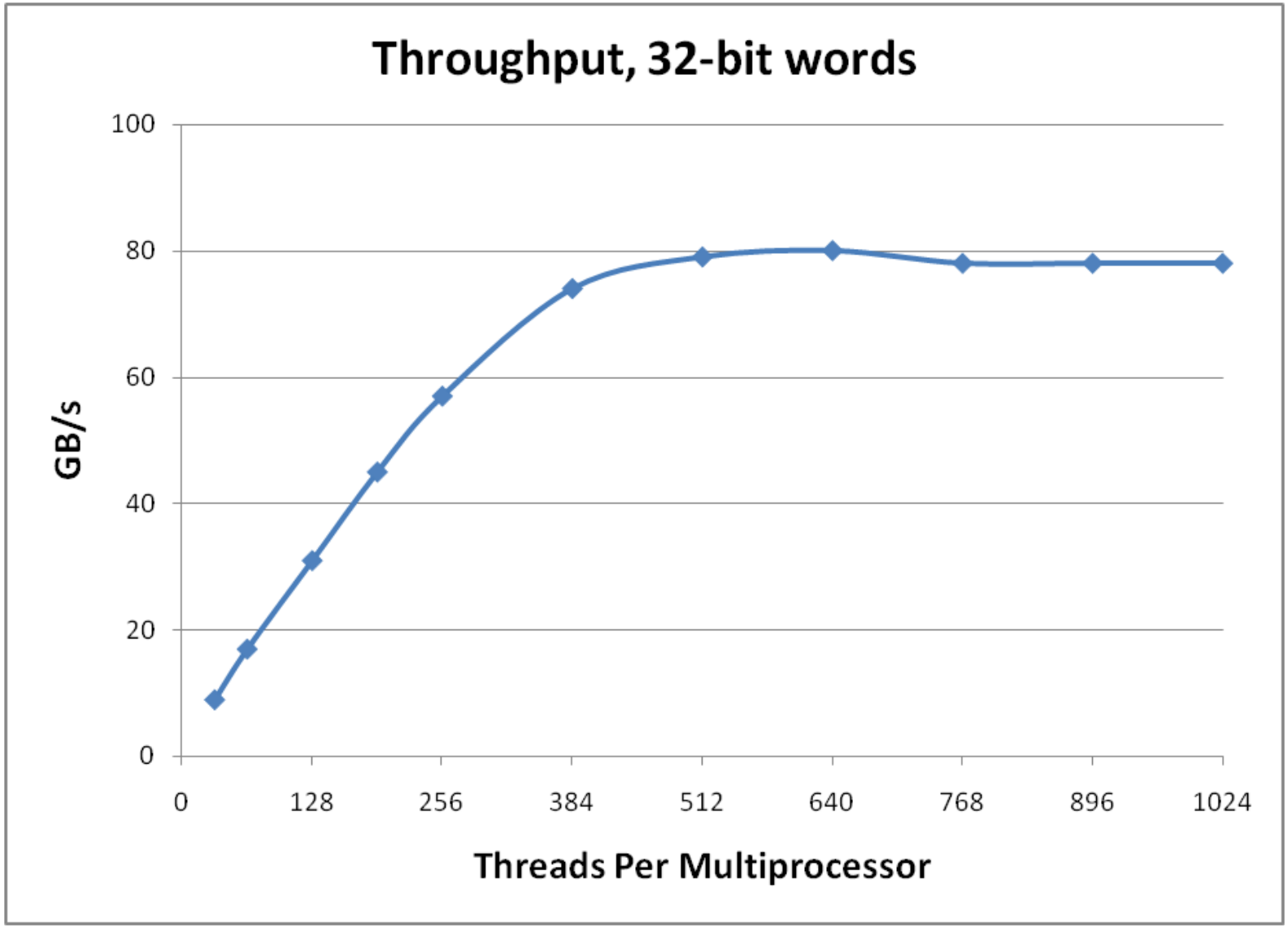
- **Pool of registers and shared memory per SM**
  - **Each thread block grabs registers & shared memory**
  - **If one or the other is fully utilized -> no more thread blocks**

Resource Limits (2)

- **Can only have 8 thread blocks per SM**
  - **If they're too small, can't fill up the SM**
  - **Need 128 threads / TB (gt200), 192 thread/ TB (gf100)**

- **Higher occupancy has diminishing returns for hiding latency**

Hiding Latency with more threads



How do you know what you're using?

- Use `nvcc -Xptxas -v` to get register and shared memory usage
  - nvcc -arch=sm_30 -Xptxas -v matrix_multiplication.cu -o mmul
- Or nvcc -arch=sm_30 --resource-usage matrix_multiplication.cu -o mmul

- Plug those numbers into CUDA Occupancy Calculator
  - Found in CUDA SDK,
    - /usr/local/cuda-7.0/tools

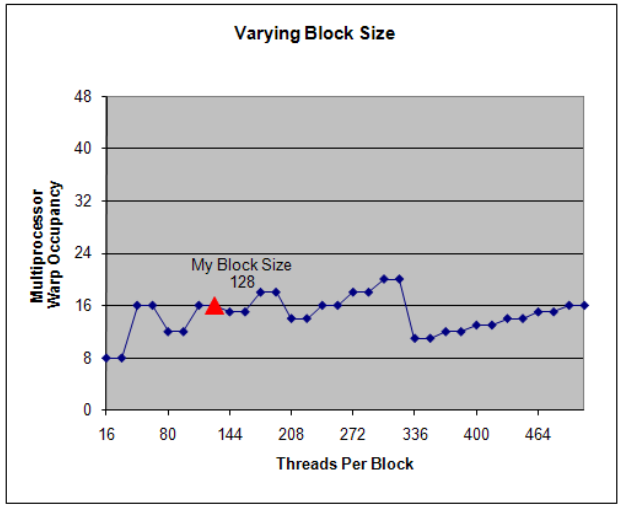
CUDA_Occupancy_calculator.xlsm - Microsoft Excel

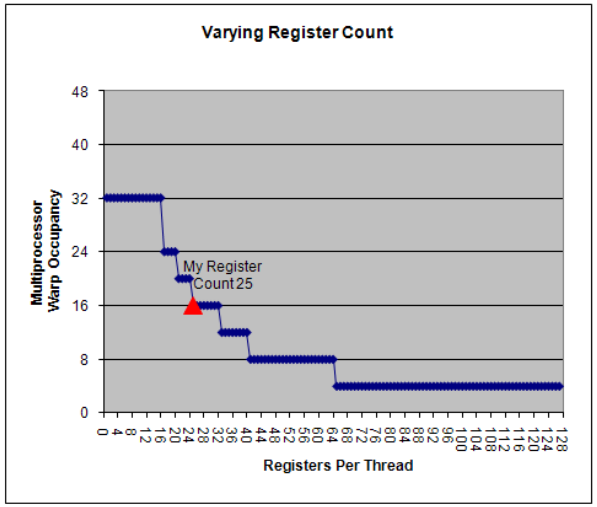
Security Warning Macros have been disabled. Options...

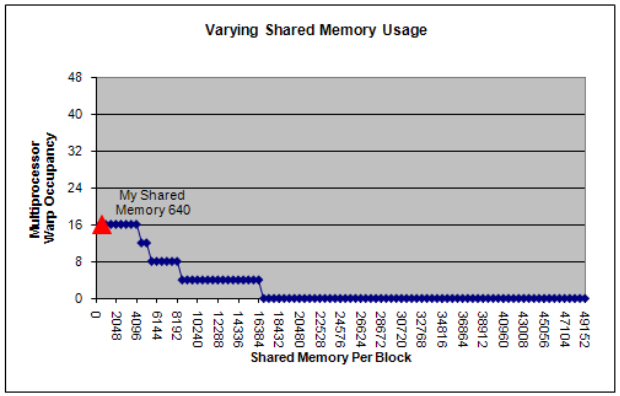
MyRegCount | 25

The other data points represent the range of possible block sizes, register counts, and shared memory allocation.

1.) Select Compute Capability (click):	**1.3**	(Help)
2.) Enter your resource usage:		
Threads Per Block	128	(Help)
Registers Per Thread	25	
Shared Memory Per Block (bytes)	640	
(Don't edit anything below this line)		
3.) GPU Occupancy Data is displayed here and in the graphs:		
Active Threads per Multiprocessor	**512**	(Help)
Active Warps per Multiprocessor	**16**	
Active Thread Blocks per Multiprocessor	**4**	
Occupancy of each Multiprocessor	**50%**	
Physical Limits for GPU Compute Capability:	**1.3**	
Threads per Warp	32	
Warps per Multiprocessor	32	
Threads per Multiprocessor	1024	
Thread Blocks per Multiprocessor	8	
Total # of 32-bit registers per Multiprocessor	16384	
Register allocation unit size	512	
Register allocation granularity	block	
Shared Memory per Multiprocessor (bytes)	16384	
Shared Memory Allocation unit size	512	
Warp allocation granularity (for register allocation)	2	
Allocation Per Thread Block		
Warps	4	
Registers	3584	
Shared Memory	1024	
These data are used in computing the occupancy data in blue		
Maximum Thread Blocks Per Multiprocessor	Blocks	
Limited by Max Warps / Blocks per Multiprocessor	8	
Limited by Registers per Multiprocessor	4	
Limited by Shared Memory per Multiprocessor	16	
Thread Block Limit Per Multiprocessor highlighted	RED	
CUDA Occupancy Calculator		
Version:	2.0	
Copyright and License		







Calculator | Help | GPU Data | Copyright & License

Ready

MySharedMemory | =5*MyThreadCount

	A	B
1	**CUDA GPU Occupancy Calcula**	
2		
3		
4	Just follow steps 1, 2, and 3 below! (or click here for help)	
5		
6	**1.) Select Compute Capability (click):**	1.3
7		
8	**2.) Enter your resource usage:**	
9	Threads Per Block	128
10	Registers Per Thread	25
11	Shared Memory Per Block (bytes)	640
12		
13	**(Don't edit anything below this line)**	
14		
15	**3.) GPU Occupancy Data is displayed here and in the graphs:**	

Calculator | Help | GPU Data | Copyright

Ready 100%

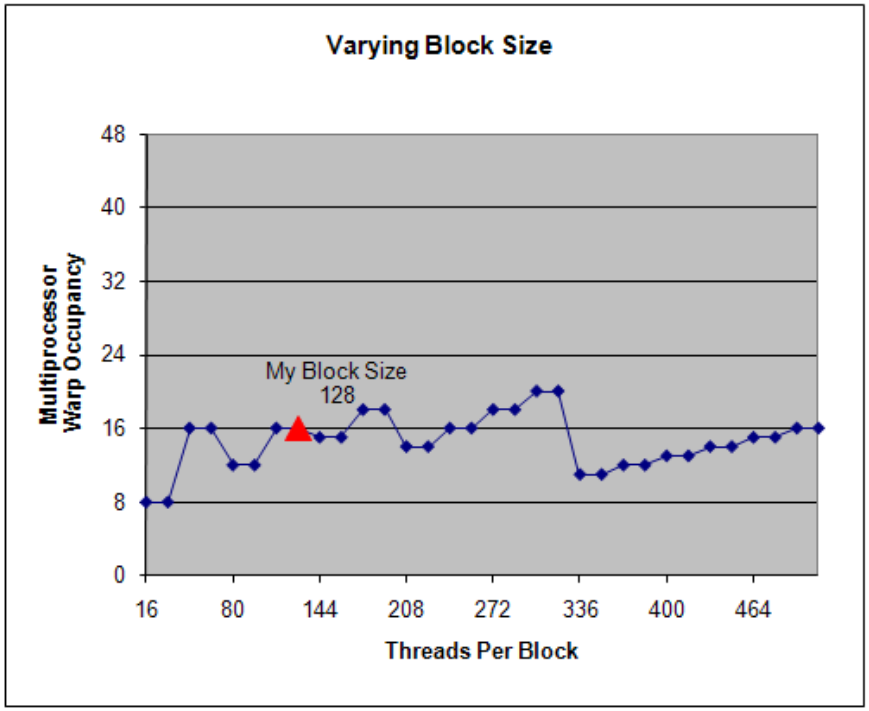
MyRegCount 23

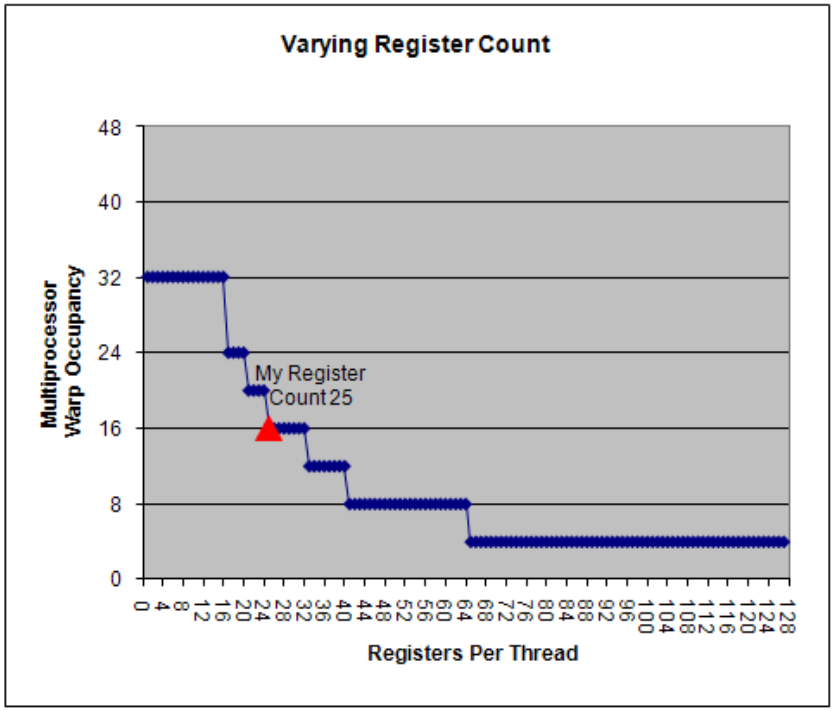
	A	B
14		
15	**3.) GPU Occupancy Data is displayed here and in the graphs:**	
16	**Active Threads per Multiprocessor**	**512**
17	**Active Warps per Multiprocessor**	**16**
18	**Active Thread Blocks per Multiprocessor**	**4**
19	**Occupancy of each Multiprocessor**	**50%**
20		
21		
22	**Physical Limits for GPU Compute Capability:**	**1.3**
23	Threads per Warp	32
24	Warps per Multiprocessor	32
25	Threads per Multiprocessor	1024
26	Thread Blocks per Multiprocessor	8
27	Total # of 32-bit registers per Multiprocessor	16384
28	Register allocation unit size	512
29	Register allocation granularity	block
30	Shared Memory per Multiprocessor (bytes)	16384
31	Shared Memory Allocation unit size	512
32	Warp allocation granularity (for register allocation)	2
33		
34	**Allocation Per Thread Block**	
35	Warps	4
36	Registers	3584
37	Shared Memory	1024
38	These data are used in computing the occupancy data in blue	
39		
40	**Maximum Thread Blocks Per Multiprocessor**	Blocks
41	Limited by Max Warps / Blocks per Multiprocessor	8
42	Limited by Registers per Multiprocessor	4
43	Limited by Shared Memory per Multiprocessor	16
44	Thread Block Limit Per Multiprocessor highlighted	RED

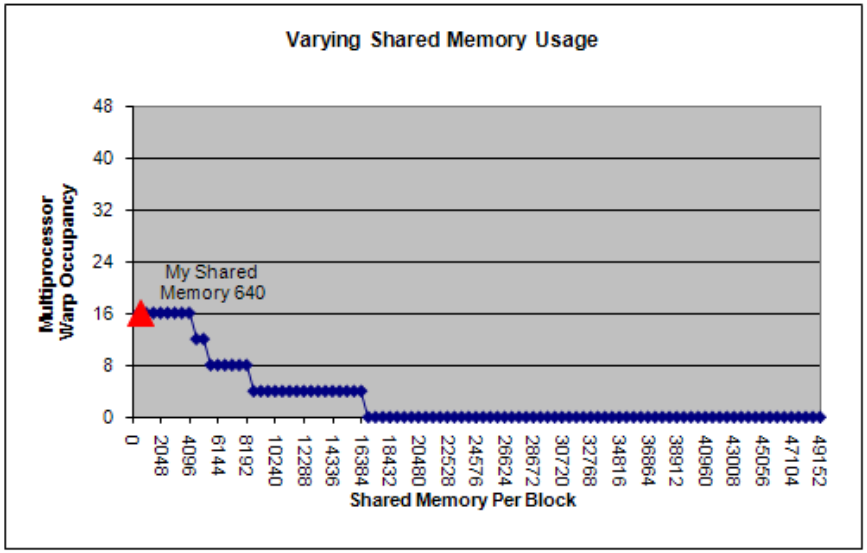
Calculator | Help | GPU Data | Co

Ready 100%

The other data points represent the range of possible block sizes, register counts, and shared memory allocation.







Calculator | Help | GPU Data | Copyright & License

KERNEL LAUNCH OVERHEAD

Kernel Launch Overhead

- Kernel launches aren't free
  - A null kernel launch will take non-trivial time
  - Actual number changes with HW generations and driver software, so I can't give you one number
- Independent kernel launches are cheaper than dependent kernel launches
  - Dependent launch: Some readback to the cpu
- If you are launching lots of small grids you will lose substantial performance due to this effect

Kernel Launch Overheads

- If you are reading back data to the cpu for control decisions, consider doing it on the GPU
- Even though the GPU is slow at serial tasks, can do surprising amounts of work before you used up kernel launch overhead

Performance Considerations

- Measure, measure, then measure some more!
- Once you identify bottlenecks, apply judicious tuning
  - What is most important depends on your program
  - You'll often have a series of bottlenecks, where each optimization gives a smaller boost than expected

Back up: Shared Memory

- Uses:
  - Inter-thread communication within a block
  - Cache data to reduce global memory accesses
  - Use it to avoid non-coalesced access
- Organization:
  - 16 banks, 32-bit wide banks (Tesla)
  - 32 banks, 32-bit wide banks (Fermi)
  - Successive 32-bit words belong to different banks
- Performance:
  - 32 bits per bank per 2 clocks per multiprocessor
  - smem accesses are per 16-threads (half-warp)
  - serialization: if *n* threads (out of 16) access the same bank, *n* accesses are executed serially
  - broadcast: *n* threads access <u>the same word</u> in one fetch

Summary

- Occupancy
  - Calculator
- Kernel Launch Overheads